

Computer Networks (CL3001)



CN LAB 08

Submitted by:

Syeda Sara Ali

23K-0070

BS-AI (6A)

Submitted to:

Sir Sameer Faisal

Lab Tasks:

1. Apply TCP Congestion control mechanism in NS3 & plot the congestion window graph.

Code.cc:

```
/* -*- Mode:C++; c-file-style:"gnu"; indent-tabs-mode:nil; -*- */
/*
 * This program is free software; you can redistribute it and/or modify
 * it under the terms of the GNU General Public License version 2 as
 * published by the Free Software Foundation;
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program; if not, write to the Free Software
 * Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA
 */

#include <fstream>
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/stats-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("SeventhScriptExample");

//
// =====
//
//
//      node 0          node 1
// +-----+ +-----+
// | ns-3 TCP | | ns-3 TCP |
// +-----+ +-----+
// | 10.1.1.1 | | 10.1.1.2 |
// +-----+ +-----+
// | point-to-point | | point-to-point |
// +-----+ +-----+
//      |           |
//      +-----+
```

```

//          5 Mbps, 2 ms
//
//
// We want to look at changes in the ns-3 TCP congestion window. We need
// to crank up a flow and hook the CongestionWindow attribute on the socket
// of the sender. Normally one would use an on-off application to generate a
// flow, but this has a couple of problems. First, the socket of the on-off
// application is not created until Application Start time, so we wouldn't be
// able to hook the socket (now) at configuration time. Second, even if we
// could arrange a call after start time, the socket is not public so we
// couldn't get at it.
//
// So, we can cook up a simple version of the on-off application that does what
// we want. On the plus side we don't need all of the complexity of the on-off
// application. On the minus side, we don't have a helper, so we have to get
// a little more involved in the details, but this is trivial.
//
// So first, we create a socket and do the trace connect on it; then we pass
// this socket into the constructor of our simple application which we then
// install in the source node.
//
=====
=====

//
class MyApp : public Application
{
public:
    MyApp ();
    virtual ~MyApp ();

    /**
     * Register this type.
     * \return The TypeId.
     */
    static TypeId GetTypeId (void);
    void Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets,
                DataRate dataRate);

private:
    virtual void StartApplication (void);
    virtual void StopApplication (void);

    void ScheduleTx (void);
    void SendPacket (void);

    Ptr<Socket>    m_socket;
    Address        m_peer;
    uint32_t       m_packetSize;
    uint32_t       m_nPackets;

```

```

    DataRate    m_dataRate;
    EventId     m_sendEvent;
    bool        m_running;
    uint32_t    m_packetsSent;
};

```

```

MyApp::MyApp ()
: m_socket (0),
  m_peer (),
  m_packetSize (0),
  m_nPackets (0),
  m_dataRate (0),
  m_sendEvent (),
  m_running (false),
  m_packetsSent (0)
{
}

```

```

MyApp::~MyApp ()
{
    m_socket = 0;
}

```

```

/* static */
TypeId MyApp::GetTypeId (void)
{
    static TypeId tid = TypeId ("MyApp")
        .SetParent<Application> ()
        .SetGroupName ("Tutorial")
        .AddConstructor<MyApp> ()
        ;
    return tid;
}

```

```

void
MyApp::Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets,
              DataRate dataRate)
{
    m_socket = socket;
    m_peer = address;
    m_packetSize = packetSize;
    m_nPackets = nPackets;
    m_dataRate = dataRate;
}

```

```

void
MyApp::StartApplication (void)
{
    m_running = true;
}

```

```

m_packetsSent = 0;
if (InetSocketAddress::IsMatchingType (m_peer))
{
    m_socket->Bind ();
}
else
{
    m_socket->Bind6 ();
}
m_socket->Connect (m_peer);
SendPacket ();
}

```

```

void
MyApp::StopApplication (void)
{
    m_running = false;

    if (m_sendEvent.IsRunning ())
    {
        Simulator::Cancel (m_sendEvent);
    }
}

```

```

if (m_socket)
{
    m_socket->Close ();
}
}

```

```

void
MyApp::SendPacket (void)
{
    Ptr<Packet> packet = Create<Packet> (m_packetSize);
    m_socket->Send (packet);

    if (++m_packetsSent < m_nPackets)
    {
        ScheduleTx ();
    }
}

```

```

void
MyApp::ScheduleTx (void)
{
    if (m_running)
    {
        Time tNext (Seconds (m_packetSize * 8 / static_cast<double> (m_dataRate.GetBitRate ())));
        m_sendEvent = Simulator::Schedule (tNext, &MyApp::SendPacket, this);
    }
}

```

```

}

static void
CwndChange (Ptr<OutputStreamWrapper> stream, uint32_t oldCwnd, uint32_t newCwnd)
{
    NS_LOG_UNCOND (Simulator::Now ().GetSeconds () << "\t" << newCwnd);
    *stream->GetStream () << Simulator::Now ().GetSeconds () << "\t" << oldCwnd << "\t" <<
        newCwnd << std::endl;
}

static void
RxDrop (Ptr<PcapFileWrapper> file, Ptr<const Packet> p)
{
    NS_LOG_UNCOND ("RxDrop at " << Simulator::Now ().GetSeconds ());
    file->Write (Simulator::Now (), p);
}

int
main (int argc, char *argv[])
{
    bool useV6 = false;

    CommandLine cmd;
    cmd.AddValue ("useIpv6", "Use Ipv6", useV6);
    cmd.Parse (argc, argv);

    NodeContainer nodes;
    nodes.Create (2);

    PointToPointHelper pointToPoint;
    pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
    pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

    NetDeviceContainer devices;
    devices = pointToPoint.Install (nodes);

    Ptr<RateErrorModel> em = CreateObject<RateErrorModel> ();
    em->SetAttribute ("ErrorRate", DoubleValue (0.00001));
    devices.Get (1)->SetAttribute ("ReceiveErrorModel", PointerValue (em));

    InternetStackHelper stack;
    stack.Install (nodes);

    uint16_t sinkPort = 8080;
    Address sinkAddress;
    Address anyAddress;
    std::string probeType;
    std::string tracePath;
    if (useV6 == false)

```

```

{
    Ipv4AddressHelper address;
    address.SetBase ("10.1.1.0", "255.255.255.0");
    Ipv4InterfaceContainer interfaces = address.Assign (devices);
    sinkAddress = InetSocketAddress (interfaces.GetAddress (1), sinkPort);
    anyAddress = InetSocketAddress (Ipv4Address::GetAny (), sinkPort);
    probeType = "ns3::Ipv4PacketProbe";
    tracePath = "/NodeList/*/ $ns3::Ipv4L3Protocol/Tx";
}
else
{
    Ipv6AddressHelper address;
    address.SetBase ("2001:0000:f00d:cafe::", Ipv6Prefix (64));
    Ipv6InterfaceContainer interfaces = address.Assign (devices);
    sinkAddress = Inet6SocketAddress (interfaces.GetAddress (1,1), sinkPort);
    anyAddress = Inet6SocketAddress (Ipv6Address::GetAny (), sinkPort);
    probeType = "ns3::Ipv6PacketProbe";
    tracePath = "/NodeList/*/ $ns3::Ipv6L3Protocol/Tx";
}

```

```

PacketSinkHelper packetSinkHelper ("ns3::TcpSocketFactory", anyAddress);
ApplicationContainer sinkApps = packetSinkHelper.Install (nodes.Get (1));
sinkApps.Start (Seconds (0.));
sinkApps.Stop (Seconds (20.));

```

```

Ptr<Socket> ns3TcpSocket = Socket::CreateSocket (nodes.Get (0),
    TcpSocketFactory::GetTypeId ());

```

```

Ptr<MyApp> app = CreateObject<MyApp> ();
app->Setup (ns3TcpSocket, sinkAddress, 1040, 1000, DataRate ("1Mbps"));
nodes.Get (0)->AddApplication (app);
app->SetStartTime (Seconds (1.));
app->SetStopTime (Seconds (20.));

```

```

AsciiTraceHelper asciiTraceHelper;
Ptr<OutputStreamWrapper> stream = asciiTraceHelper.CreateFileStream ("seventh.cwnd");
ns3TcpSocket->TraceConnectWithoutContext ("CongestionWindow", MakeBoundCallback
    (&CwndChange, stream));

```

```

PcapHelper pcapHelper;
Ptr<PcapFileWrapper> file = pcapHelper.CreateFile ("seventh.pcap", std::ios::out,
    PcapHelper::DLT_PPP);
devices.Get (1)->TraceConnectWithoutContext ("PhyRxDrop", MakeBoundCallback
    (&RxDrop, file));

```

```

// Use GnuplotHelper to plot the packet byte count over time
GnuplotHelper plotHelper;

```

```

// Configure the plot. The first argument is the file name prefix
// for the output files generated. The second, third, and fourth

```

```

// arguments are, respectively, the plot title, x-axis, and y-axis labels
plotHelper.ConfigurePlot ("seventh-packet-byte-count",
    "Packet Byte Count vs. Time",
    "Time (Seconds)",
    "Packet Byte Count");

// Specify the probe type, trace source path (in configuration namespace), and
// probe output trace source ("OutputBytes") to plot. The fourth argument
// specifies the name of the data series label on the plot. The last
// argument formats the plot by specifying where the key should be placed.
plotHelper.PlotProbe (probeType,
    tracePath,
    "OutputBytes",
    "Packet Byte Count",
    GnuplotAggregator::KEY_BELOW);

// Use FileHelper to write out the packet byte count over time
FileHelper fileHelper;

// Configure the file to be written, and the formatting of output data.
fileHelper.ConfigureFile ("seventh-packet-byte-count",
    FileAggregator::FORMATTED);

// Set the labels for this formatted output file.
fileHelper.Set2dFormat ("Time (Seconds) = %.3e\tPacket Byte Count = %.0f");

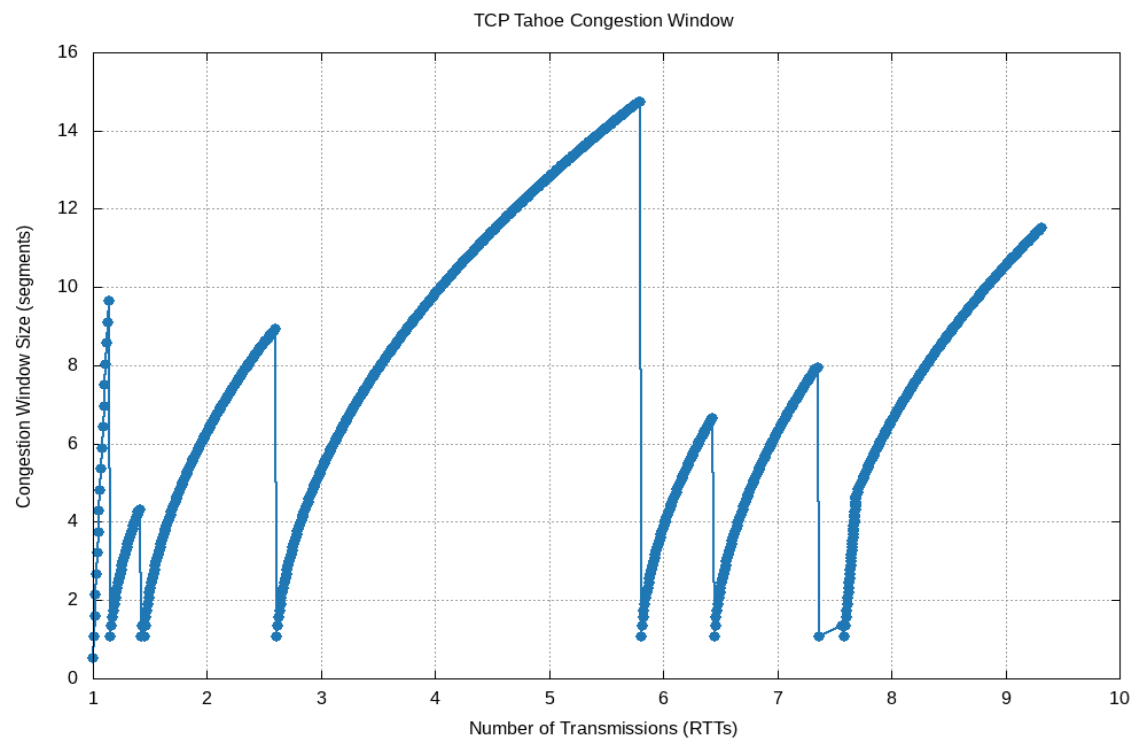
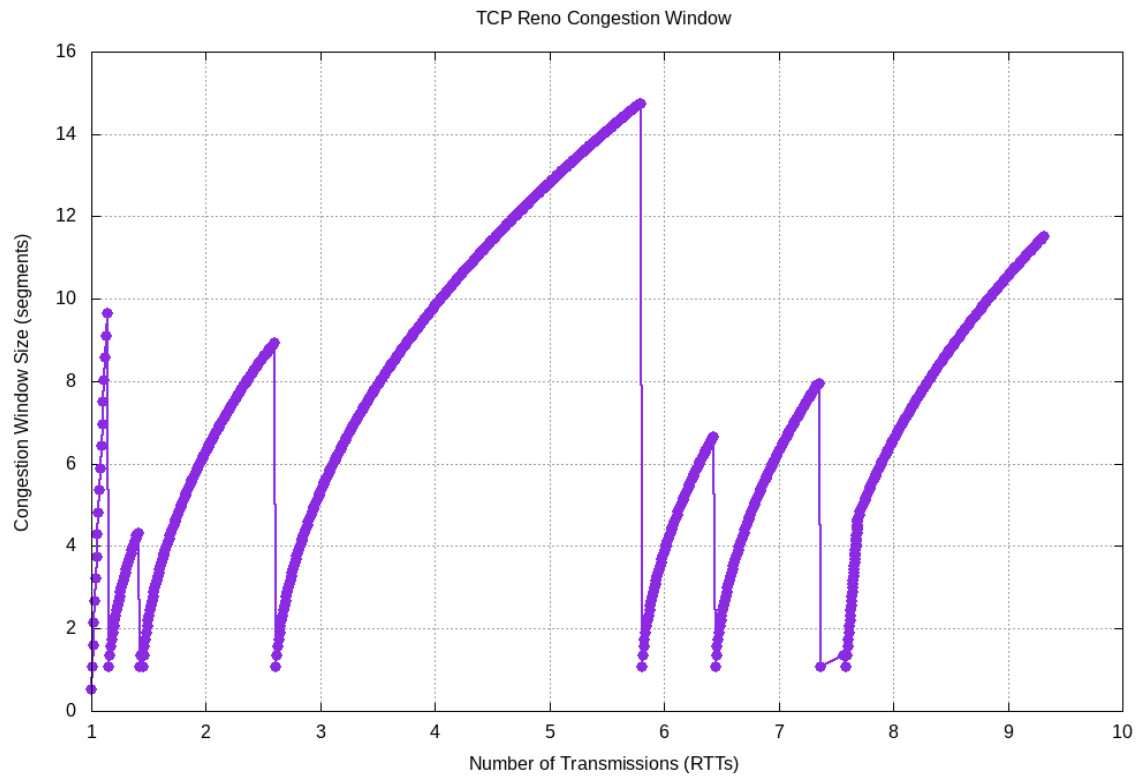
// Specify the probe type, trace source path (in configuration namespace), and
// probe output trace source ("OutputBytes") to write.
fileHelper.WriteProbe (probeType,
    tracePath,
    "OutputBytes");

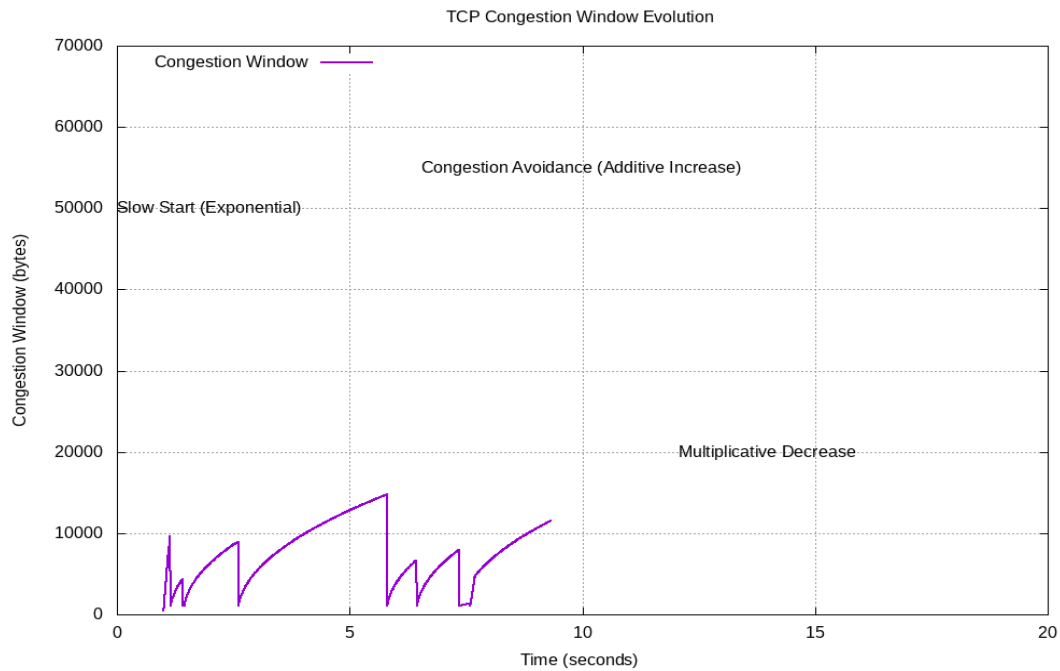
Simulator::Stop (Seconds (20));
Simulator::Run ();
Simulator::Destroy ();

return 0;
}

```


Output:





Code.plt:

1.

```
set terminal png size 1000,700 enhanced
set output "tcp_reno.png"
```

```
set title "TCP Reno Congestion Window"
set xlabel "Number of Transmissions (RTTs)"
set ylabel "Congestion Window Size (segments)"
```

```
set grid
set key off
```

```
set style line 1 lc rgb '#8A2BE2' lw 2 pt 7 ps 1.2
```

```
# Threshold lines (example)
set arrow from 5,32 to 15,32 nohead dt 2 lw 2
set label "sssthresh" at 6,34
```

```
set arrow from 10,22 to 20,22 nohead dt 2 lw 2
set label "sssthresh" at 11,24
```

```
plot "seventh.cwnd" using 1:($3/1000) with linespoints ls 1
```

2.

```
set terminal png size 1000,700 enhanced
```

```

set output "tcp_tahoe.png"

set title "TCP Tahoe Congestion Window"
set xlabel "Number of Transmissions (RTTs)"
set ylabel "Congestion Window Size (segments)"

set grid
set key off

set style line 1 lc rgb '#1f77b4' lw 2 pt 7 ps 1.2

# Optional threshold line
set arrow from 5,32 to 15,32 nohead dt 2 lw 2
set label "ssthresh" at 6,34

# Plot (bytes → segments)
plot "seventh.cwnd" using 1:($3/1000) with linespoints ls 1

3.
# Gnuplot script for TCP Congestion Window Graph
# Corrected syntax for older gnuplot versions

set terminal png size 1000,700 enhanced
set output "tcp_congestion_window.png"

# Set style for quality graph
set style line 1 lc rgb '#0060ad' lt 1 lw 3
set style line 2 lc rgb '#dd181f' lt 2 lw 2

set title "TCP Congestion Window Evolution"
set xlabel "Time (seconds)"
set ylabel "Congestion Window (bytes)"

set grid
set key top left

# Set axis ranges (adjust based on your data)
set xrange [0:20]
set yrange [0:70000]

# Add phase annotations
set label 1 "Slow Start (Exponential)" at 2,50000 center
set label 2 "Congestion Avoidance (Additive Increase)" at 10,55000 center
set label 3 "Multiplicative Decrease" at 14,20000 center

# Plot the congestion window
# Try different possible filenames
plot "seventh.cwnd" using 1:3 with lines linewidth 2 title "Congestion Window"

```